DualMS 2.0: Practical Lattice-Based Two-Round Fiat–Shamir Multi-Signature with Better Efficiency

Artifact Appendix

Qiqi Lai
Shaanxi Normal University
Xi'an, China

Chongshen Chen
Shaanxi Normal University
Xi'an, China

Feng-Hao Liu
Washington State University
Pullman, USA

Tianyu Zhao
Washington State University
Pullman, USA

Qi Wang
Shanghai Jiao Tong University
Shanghai, China

Zhedong Wang
Shanghai Jiao Tong University
Shanghai, China

A Artifact Appendix

A.1 Abstract

This artifact accompanies the paper "DualMS 2.0: Practical Lattice-Based Two-Round Fiat–Shamir Multi-Signature with Better Efficiency" (CCS 2026, B Submission 303). It packages (i) SageMath scripts that regenerate the parameter and size results in Tables 1, 6, and 8; (ii) Go implementations and prebuilt Linux AMD64 binaries for DualMS and rounded DualMS 2.0; (iii) scripts and logs for the local (Table 7) and WAN (Table 8) benchmarks. This is a proof-of-concept artifact only, not for production use.

A.2 Description & Requirements

The top-level layout is:

- `README.md`: usage, hardware notes, and how to run each experiment.
- `sage/`: `table1.sage`, `table6.sage`, `table8.sage`, scheme data, and helpers.
- `dualms-benchmark/` and `dualms2.0-benchmark/`:
  - Go source under `internal/`
  - prebuilt binaries under `bin/`
  - local benchmarks under `internal/local/`
  - WAN materials under `network/` (JSON templates, runner scripts, and `wan-performance-table8/` summaries)

A.2.1 Security, privacy, and ethical concerns. This is a proof-of-concept and should not be used for real-world deployment; it raises no security, privacy, or ethical concerns.

A.2.2 How to access. The artifact is available at https://doi.org/10.5281/zenodo.22344990.

A.2.3 Hardware dependencies.

- **Local / Sage:** Linux AMD64 recommended. Paper local timings used Ubuntu 24.04.3 on an AMD Ryzen 7 6800H (8 cores, up to 4.3 GHz) with 32 GB RAM, single core.
- **WAN:** one coordinator host server and remote signer hosts server reachable over the public Internet (see WAN note below). Exact latency depends on geography and network conditions.

A.2.4 Software dependencies.

- Linux AMD64 (Ubuntu 24.04.3).
- Go ≥1.19; Lattigo v5.0.2 (`go.mod`).
- SageMath with Python 3.

A.2.5 Benchmarks. No external datasets are required: experiments use the paper's parameters, and results and logs are stored locally in the artifact.

A.3 Set Up

A.3.1 Installation.

(1) Unpack the artifact archive and `cd` into the top-level directory.
(2) Install SageMath and Go ≥1.19.
(3) Prebuilt binaries are under `dualms-benchmark/bin/dualms` and `dualms2.0-benchmark/bin/sec3rounding`. To rebuild:

```
cd dualms-benchmark
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" \
  -o bin/dualms .
cd ../dualms2.0-benchmark
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" \
  -o bin/sec3rounding .
```

A.3.2 Basic test.

```
cd sage && sage table1.sage
cd ../dualms-benchmark && ./bin/dualms bench 5 32
```

Expected: Sage prints a Table 1 size comparison; the Go bench prints multi-signature running performance.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1): DualMS 2.0 size results compared to DualMS as summarized in Table 1 and Table 6.
(C2): Local benchmarks of DualMS 2.0 and DualMS presented in Table 7.
(C3): WAN network benchmarks of DualMS 2.0 and DualMS for n=8 signers presented in Table 8. See the WAN experiment note below.

A.4.2 Experiments.

(E1): **Size / bit security levels (Tables 1 and 6)**:
Preparation: Install requirements listed in Section A.3.
Execution: Run
`./run-menu.sh`
and choose `[1]` (Tables 1 & 6); pick Table 1, Table 6, or both.
Results: Outputs should match Tables 1 and 6.

(E2): **Local benchmark (Table 7)**:
Preparation: Install requirements listed in Section A.3.

Execution: Run `./run-menu.sh` and choose `[2]`; select DualMS / DualMS 2.0 with n=32 or 1024, or run all four.
Results: Compare latencies to Table 7. Absolute numbers vary by CPU; relative DualMS vs. DualMS 2.0 trends should align.

(E3): **WAN summary (Table 8)**:
Preparation: Install requirements listed in Section A.3.
Execution: Run `./run-menu.sh` and choose `[3]` then `[1]` to format the packaged WAN summaries into Table 8 from each benchmark's `network/wan-performance-table8/table8-summary.json`.
Results: WAN experiment note. The Netherlands VPS used in the paper's Table 8 WAN setup (signers 4 and 5) has expired and is no longer available. Therefore, the WAN experiment results of the DualMS scheme come from a later, separate experiment: the Netherlands signer server was dropped, and signers were redistributed to HK (coordinator + signer 1), US1 (signers 2–4), and UAE (signers 5–7). Latency and traffic thus differ from Table 8. Local benchmarks remain reproducible via the menu above.

A.5 Version

Based on the LaTeX template for Artifact Evaluation V20220926.